Lab 07
OpenGL Hierarchical Modeling

Objectives

- We will learn the basic of Hierarchical Modeling in OpenGL

Programming Exercises (90 minutes)

The general rule in modelling a complex model is that you should break a model into smaller components and model the components individually. Each component should be modelled in its own coordinate system with the origin of the coordinate system being (0,0,0). When modelling a component, you would probably need to use the `glTranslatef(...)`, `glRotationf(...)` and `glScalef(...)` transformations to modify the **MODELVIEW matrix**. In order that subsequence objects can be modelled correctly, you should save the original MODELVIEW matrix before you carry out any transformations and restore back the original MODELVIEW matrix once you are done. The best way to do so, as shown in Lab 05 is to call the `glPushMatrix()` function before the beginning of your modelling code and `glPopMatrix()` function after you have done all your modelling.

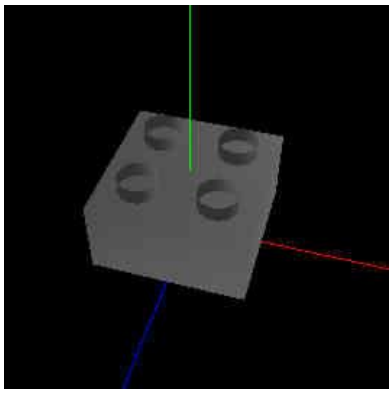
The notion of modelling each component of a particular object at its own coordinate system and the whole object is modelled as a combination and hierarchy of objects is called **Hierarchical Modelling**. In OpenGL, this is primarily achieved using the `glPushMatrix()` and `glPopMatrix()` function. One example is that when we model a robot, we would wish to model the head relative to the body but when we are modelling the eyes, it would be better for us to model them relative to the head instead of the body.

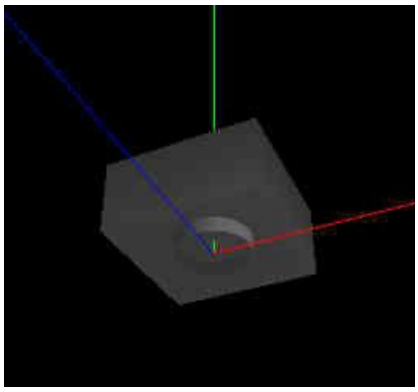
Task 1: Bottom-up Modeling *(30 minutes)*

Task 1A: Setting up

Modelling in OpenGL and computer graphics in general is pretty much like building a model using Lego set, you build smaller components and join them up to form a big model. In fact, what better way to show hierarchical modelling than to show how to build our own virtual Lego set!!

Before we could build anything with our virtual Lego set, we need to build our Lego pieces first, we will start by building the most basic piece as shown below in two different view.





As you can see, we will model this basic Lego piece at the origin (0,0,0) and the origin is at the centre of the base of the piece. There are 3 different components, namely the 4 cylinders on top, the cylinder at the bottom and the cube itself. For the cylinders, we will use the Quadric provided by OpenGL's GLU library as shown in Lab 05.

Let's first declare the `MyLego` class. Part of `CGLab07.hpp` should have these lines below:

```
class MyLego
{
 public:
    MyLego();
    ~MyLego();

    void draw();

    //functions to draw Lego pieces
    void draw2x2Lego();
    void draw2x4Lego();
    void draw4x4Lego();
    void draw2x8Lego();
    void draw8x8Lego();

    //testing functions, we will build this one by one
    void draw1();
    void draw2();
    void drawOneTower();
    void drawTowers();

 private:
    //bulding blocks for the basic Lego piece
    void drawTopCylinders();
    void drawBottomCylinder();
    void drawCube();
    GLUquadricObj *pObj;
};
```

The `CGLab07.cpp` is given as below:

```
/*
 TCG6223 Computer Graphics
 FIST, Multimedia University

 CGLab07.cpp

 Objective: Lab07 on Hierarchical Modeling
*/

#include <GL/glut.h>
#include "CGLabmain.hpp"
#include "CGLab07.hpp"

using namespace CGLab07;

MyLego::MyLego()
{
 //Setup Quadric Object
 pObj = gluNewQuadric();
 gluQuadricNormals(pObj, GLU_SMOOTH);
}
```

```
MyLego::~MyLego()
{
 gluDeleteQuadric(pObj);
}

void MyLego::draw()
{
 glDisable(GL_CULL_FACE);

 //remark and un-remark accordingly to run the
 //   various testing functions as you build them
 draw1();
 draw2();

 glColor3f(1.0f, 1.0f, 1.0f);
 draw2x2Lego();

 draw2x4Lego();
 draw4x4Lego();
 draw2x8Lego();
 draw8x8Lego();
 drawOneTower();
 drawTowers();

 glEnable(GL_CULL_FACE);
}

//draw a lego piece with dimension 2.0 x 1.0 x 2.0,
//  the bottom center of the lego piece is at 0,0,0
void MyLego::drawCube()
{
 GLfloat dimx = 2.0, dimy = 1.0, dimz = 2.0;
 GLfloat xmax = dimx/2.0;
 GLfloat xmin = -xmax;
 GLfloat ymax = dimy;
 GLfloat ymin = 0;
 GLfloat zmax = dimz/2.0;
 GLfloat zmin = -zmax;

 //Box
 glBegin(GL_QUADS);
    //front
    glVertex3f(xmin,ymax,zmax);
    glVertex3f(xmin,ymin,zmax);
    glVertex3f(xmax,ymin,zmax);
    glVertex3f(xmax,ymax,zmax);
    //back
    glVertex3f(xmin,ymax,zmin);
    glVertex3f(xmax,ymax,zmin);
    glVertex3f(xmax,ymin,zmin);
    glVertex3f(xmin,ymin,zmin);
    //right
    glVertex3f(xmax,ymax,zmax);
    glVertex3f(xmax,ymin,zmax);
    glVertex3f(xmax,ymin,zmin);
    glVertex3f(xmax,ymax,zmin);
    //left
    glVertex3f(xmin,ymax,zmax);
    glVertex3f(xmin,ymax,zmin);
    glVertex3f(xmin,ymin,zmin);
```

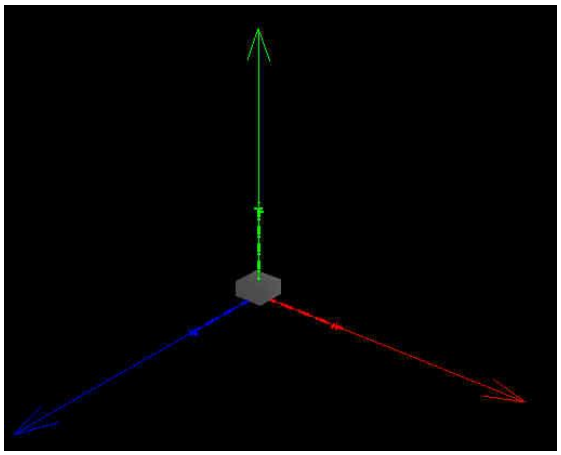
```
    glVertex3f(xmin,ymin,zmax);
    //top
    glVertex3f(xmin,ymax,zmax);
    glVertex3f(xmax,ymax,zmax);
    glVertex3f(xmax,ymax,zmin);
    glVertex3f(xmin,ymax,zmin);
    //bottom
    glVertex3f(xmin,ymin+0.2,zmax);
    glVertex3f(xmax,ymin+0.2,zmax);
    glVertex3f(xmax,ymin+0.2,zmin);
    glVertex3f(xmin,ymin+0.2,zmin);
 glEnd();
}

void MyLego::drawBottomCylinder()
{
 // fill later...
}

void MyLego::drawTopCylinders()
{
 // fill later...
}

void MyLego::draw2x2Lego()
{
 // fill later...
}

void MyLego::draw2x4Lego()
{
 // fill later...
}

void MyLego::draw4x4Lego()
{
 // fill later...
}

void MyLego::draw2x8Lego()
{
 // fill later...
}

void MyLego::draw8x8Lego()
{
 // fill later...
}

void MyLego::draw1()
{
 MyAxis axis;
 axis.setLength(6.0f, 6.0f, 6.0f);
 axis.setLineWidth(3);
 axis.setLineStipple(1, 0xff60);

 axis.draw();
 glColor3f(1.0f, 1.0f, 1.0f);
 drawCube();
}
```

```
void MyLego::draw2()
{
 // fill later...
}

void MyLego::drawOneTower()
{
 // fill later...
}

void MyLego::drawTowers()
{
 // fill later...
}
```

The `MyLego::drawCube()` function as shown above has been taught earlier, just that now we build a cube of size 2.0 x 1.0 x 2.0 and the base which is modelled at (0,0,0) is the bottom centre of the cube. **NOTE:** It is important to model objects with its base centred at the origin, so that we have a clear reference point for the objects when they undergo transformations later on.

Take note that we have filled in the `MyLego::draw1()` function which calls the `MyLego::drawCube()` function. You should run the program to make sure it works.

The output that you should see is something as below:



Take note of the additional axes (which is originally defined in `CGLabmain.hpp`) that we have attached to show the object coordinate system (i.e. model coordinate system) of the cube. Here we use a stipple line style to differentiate it from the world coordinate system. Note that by default, we use the colors, red, green and blue for the x-axis, y-axis and z-axis respectively.

Task 1B: Building the components

We will continue to build the remaining two components that form the basic Lego piece.

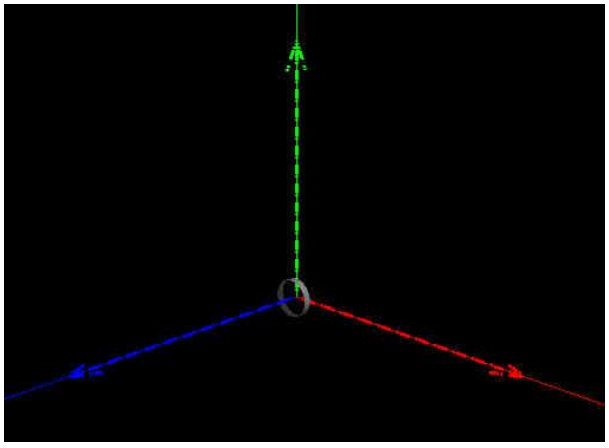
The next thing to build is the cylinder at the bottom, the function to build is as below:

```
void MyLego::drawBottomCylinder()
{
 gluCylinder(pObj, 0.5f, 0.5f, 0.2f, 26, 13);
}
```

Then, we modify the `MyLego::draw1()` function accordingly to include the bottom cylinder:

```
void MyLego::draw1()
{
 MyAxis axis;
 axis.setLength(6.0f, 6.0f, 6.0f);
 axis.setLineWidth(3);
 axis.setLineStipple(1, 0xff60);

 axis.draw();
 glColor3f(1.0f, 1.0f, 1.0f);
 //drawCube();  //remark this for the time being
 drawBottomCylinder();
}
```

The output is as below:



Take note that by default, **the OpenGL's cylinder is oriented parallel to the z-axis**. It does not matter for the time being, we will rotate it to the proper place (below the cube base) when we combine this with the rest of the components to form the Lego piece.

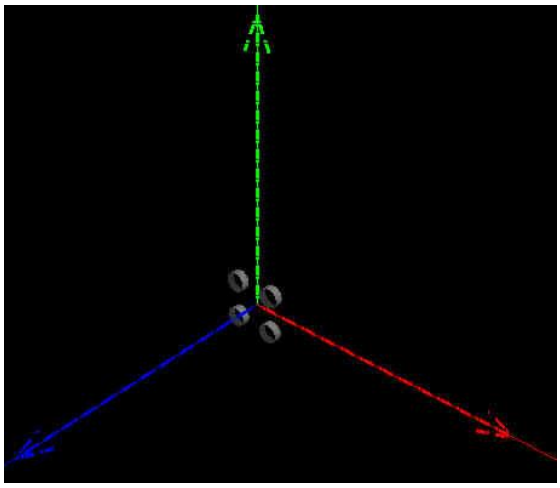
Now we are ready to build the 3rd component, which are the 4 cylinders at the top of the Lego piece. The code would be as below:

```
void MyLego::drawTopCylinders()
{
 glPushMatrix();
    glTranslatef(-0.5f,-0.5f, 0.0f);
    gluCylinder(pObj, 0.25f, 0.25f, 0.2f, 26, 13);
    glTranslatef( 1.0f, 0.0f, 0.0f);
    gluCylinder(pObj, 0.25f, 0.25f, 0.2f, 26, 13);
    glTranslatef( 0.0f, 1.0f, 0.0f);
    gluCylinder(pObj, 0.25f, 0.25f, 0.2f, 26, 13);
    glTranslatef(-1.0f, 0.0f, 0.0f);
    gluCylinder(pObj, 0.25f, 0.25f, 0.2f, 26, 13);
 glPopMatrix();
}
```

Again, we modify the `MyLego::draw1()` function accordingly to include the 4 top cylinders:

```
void MyLego::draw1()
{
 MyAxis axis;
 axis.setLength(6.0f, 6.0f, 6.0f);
 axis.setLineWidth(3);
 axis.setLineStipple(1, 0xff60);

 axis.draw();
 glColor3f(1.0f, 1.0f, 1.0f);
 //drawCube();  //remark this for the time being
 //drawBottomCylinder(); //remark this for the time being
 drawTopCylinders();
}
```

The output is as below:



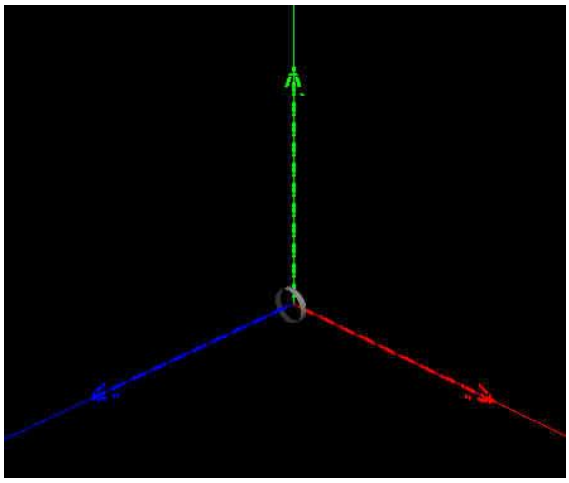
Again as you can see, we modelled it at the x-y plane, this is because the default orientation of cylinder is parallel to the z-axis. Take note that we also model it centred at the origin, this is not a must but it is a good practice to have a clear reference point for an object.

Task 1C: Putting the components together

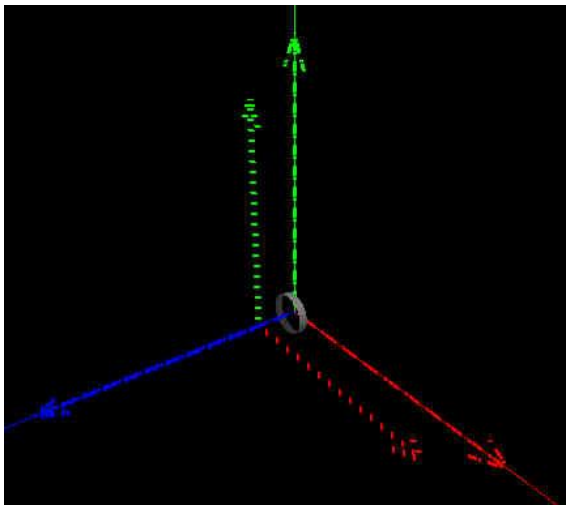
Now that we have all the required components to build a Lego piece, we will demonstrate how we can put them together to form the piece. Since each of these components has its own local coordinate system (i.e. model coordinate system), we will need to perform the necessary transformation to put them into their correct positions and orientations.

Below is a table showing each step of how this can be achieved. The output for each step is also shown for clearer illustration.

```
void MyLego::draw2()
{
 MyAxis axis1;
 axis1.setLength(6.0f, 6.0f, 6.0f);
 axis1.setLineWidth(3);
 axis1.setLineStipple(1, 0xff60);
 MyAxis axis2;
 axis2.setLength(5.0f, 5.0f, 5.0f);
 axis2.setLineWidth(5);
 axis2.setLineStipple(1, 0x6060);

 axis1.draw();
 glColor3f(1.0f, 1.0f, 1.0f);
 drawBottomCylinder();
}
```

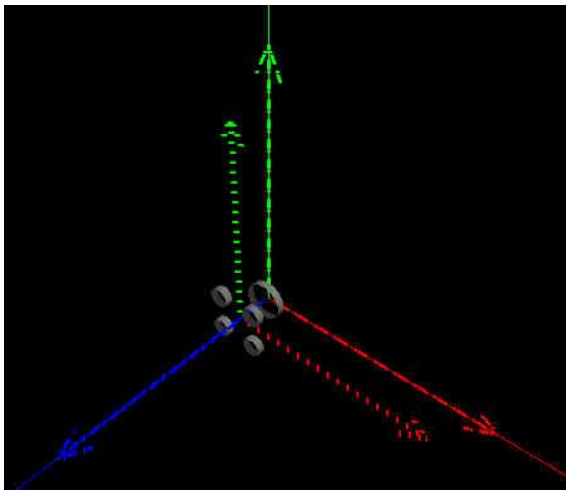


```
void MyLego::draw2()
{
 MyAxis axis1;
 axis1.setLength(6.0f, 6.0f, 6.0f);
 axis1.setLineWidth(3);
 axis1.setLineStipple(1, 0xff60);
 MyAxis axis2;
 axis2.setLength(5.0f, 5.0f, 5.0f);
 axis2.setLineWidth(5);
 axis2.setLineStipple(1, 0x6060);

 axis1.draw();
 glColor3f(1.0f, 1.0f, 1.0f);
 drawBottomCylinder();

 glColor3f(1.0f, 1.0f, 1.0f);
 glTranslatef(0.0f, 0.0f, 1.0f);
 axis2.draw();
}
```



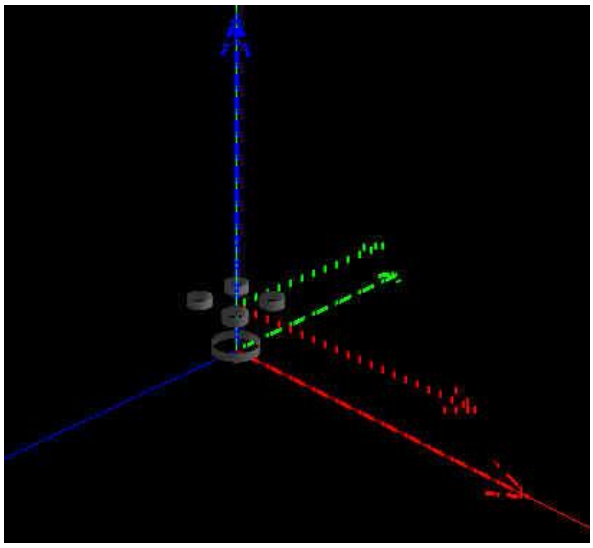
The current local coordinate system has been shifted as shown.

```
void MyLego::draw2()
{
 MyAxis axis1;
 axis1.setLength(6.0f, 6.0f, 6.0f);
 axis1.setLineWidth(3);
 axis1.setLineStipple(1, 0xff60);
 MyAxis axis2;
 axis2.setLength(5.0f, 5.0f, 5.0f);
 axis2.setLineWidth(5);
 axis2.setLineStipple(1, 0x6060);

 axis1.draw();
 glColor3f(1.0f, 1.0f, 1.0f);
 drawBottomCylinder();

 glColor3f(1.0f, 1.0f, 1.0f);
 glTranslatef(0.0f, 0.0f, 1.0f);
 axis2.draw();

 glColor3f(1.0f, 1.0f, 1.0f);
 drawTopCylinders();
}
```



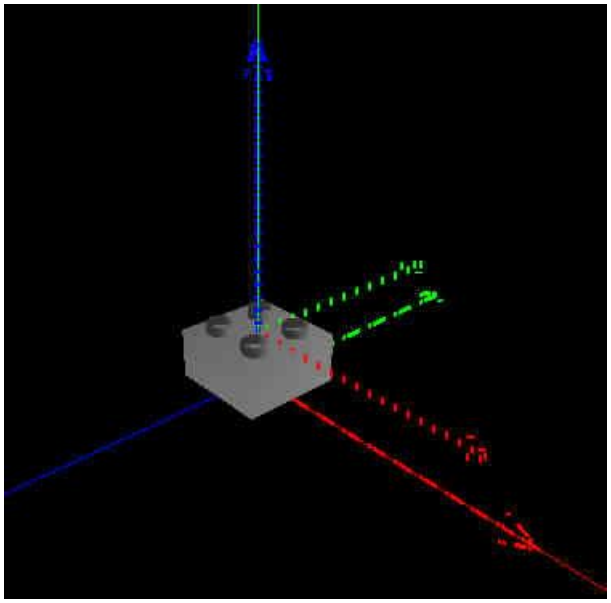
Since the current local coordinate system has been shifted, whatever that is being drawn would be drawn relative to this current local coordinate system.

```
void MyLego::draw2()
{
 MyAxis axis1;
 axis1.setLength(6.0f, 6.0f, 6.0f);
 axis1.setLineWidth(3);
 axis1.setLineStipple(1, 0xff60);
 MyAxis axis2;
 axis2.setLength(5.0f, 5.0f, 5.0f);
 axis2.setLineWidth(5);
 axis2.setLineStipple(1, 0x6060);

 glRotatef(-90.0f,1.0f,0.0f,0.0f);

 axis1.draw();
 glColor3f(1.0f, 1.0f, 1.0f);
 drawBottomCylinder();

 glColor3f(1.0f, 1.0f, 1.0f);
 glTranslatef(0.0f, 0.0f, 1.0f);
 axis2.draw();

 glColor3f(1.0f, 1.0f, 1.0f);
 drawTopCylinders();
}
```



The current local coordinate system is now rotated 90 degrees CW about the x-axis, thus the z-axis of the current local coordinate system is now parallel to the y-axis of the world coordinates.

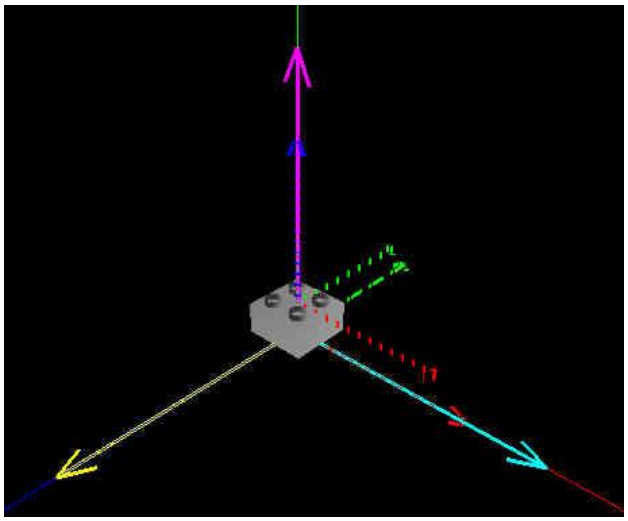
```
void MyLego::draw2()
{
 MyAxis axis1;
 axis1.setLength(6.0f, 6.0f, 6.0f);
 axis1.setLineWidth(3);
 axis1.setLineStipple(1, 0xff60);
 MyAxis axis2;
 axis2.setLength(5.0f, 5.0f, 5.0f);
 axis2.setLineWidth(5);
 axis2.setLineStipple(1, 0x6060);

 glColor3f(1.0f, 1.0f, 1.0f);
 drawCube();

 glRotatef(-90.0f,1.0f,0.0f,0.0f);

 axis1.draw();
 glColor3f(1.0f, 1.0f, 1.0f);
 drawBottomCylinder();

 glColor3f(1.0f, 1.0f, 1.0f);
 glTranslatef(0.0f, 0.0f, 1.0f);
 axis2.draw();

 glColor3f(1.0f, 1.0f, 1.0f);
 drawTopCylinders();
}
```



Here we backtrack our steps a little, to draw the cube before the change of the current local coordinate system.

IMPORTANT NOTE: From the output here, you will notice that the current local coordinate system has its y-axis pointing towards the negative z-axis of the world coordinates; its z-axis is pointing parallel to the y-axis of the world coordinates. Thus, whatever that would be drawn after this would be drawn relative to this current local coordinate system.

```
void MyLego::draw2()
{
 MyAxis axis1;
 axis1.setLength(6.0f, 6.0f, 6.0f);
 axis1.setLineWidth(3);
 axis1.setLineStipple(1, 0xff60);
 MyAxis axis2;
 axis2.setLength(5.0f, 5.0f, 5.0f);
 axis2.setLineWidth(5);
 axis2.setLineStipple(1, 0x6060);

 glPushMatrix();

 glColor3f(1.0f, 1.0f, 1.0f);
 drawCube();

 glRotatef(-90.0f,1.0f,0.0f,0.0f);

 axis1.draw();
 glColor3f(1.0f, 1.0f, 1.0f);
 drawBottomCylinder();

 glColor3f(1.0f, 1.0f, 1.0f);
 glTranslatef(0.0f, 0.0f, 1.0f);
 axis2.draw();

 glColor3f(1.0f, 1.0f, 1.0f);
 drawTopCylinders();

 glPopMatrix();

 MyAxis axis3;
 axis3.setXColor(0.0f,1.0f,1.0f);
 axis3.setYColor(1.0f,0.0f,1.0f);
 axis3.setZColor(1.0f,1.0f,0.0f);
 axis3.setLength(8.0f,8.0f,8.0f);
 axis3.setLineWidth(3);
 axis3.draw();
}
```



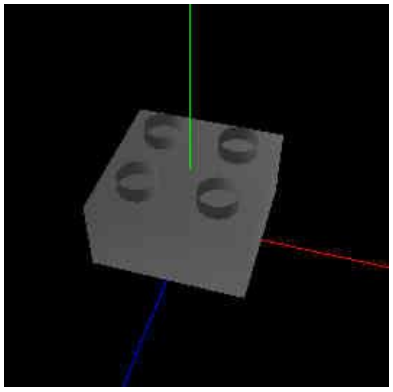
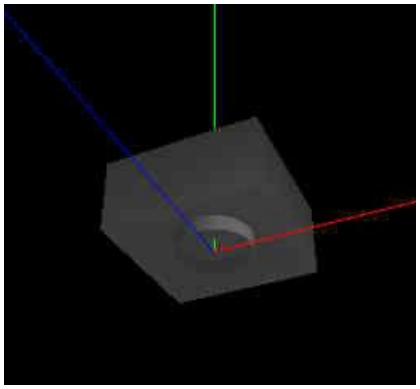
To ensure that the current local coordinate system remain the same before and after our modelling, it is a good practice to save the **MODELVIEW matrix** by using `glPushMatrix()` before we start our modelling and restore it back using `glPopMatrix()` after we are done.

Take note that after adding these codes, our current local coordinate system is back to as before, as illustrated by the thick axes of cyan, magenta and yellow colors.

To put all these together, we now have a function to draw a single piece (the width and length are 2.0 but the height is 1.0) of the basic Lego piece as below:

```
void MyLego::draw2x2Lego()
{
 glPushMatrix();
    drawCube();
    glRotatef(-90.0f, 1.0f, 0.0f, 0.0f);
    drawBottomCylinder();
    glTranslatef(0.0f, 0.0f, 1.0f);
    drawTopCylinders();
 glPopMatrix();
}
```

After doing necessary modifications (by commenting out previous functions calls) to the `MyLego::draw()` function, we would get the output as below (viewed from two different views):

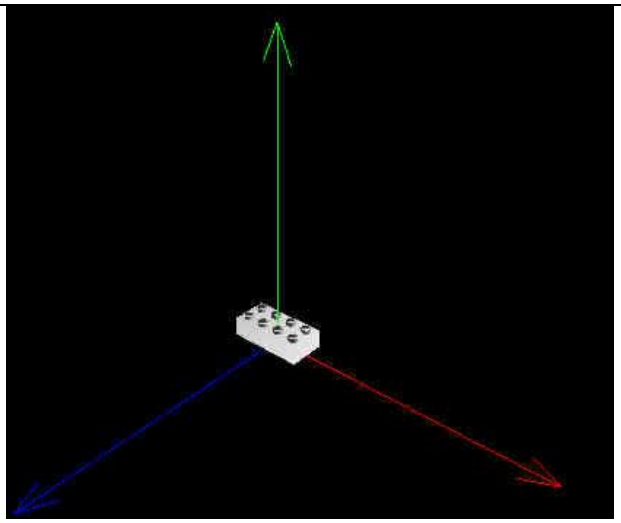



Task 2: Modelling with reusable components *(60 minutes)*

Task 2A: Using reusable components

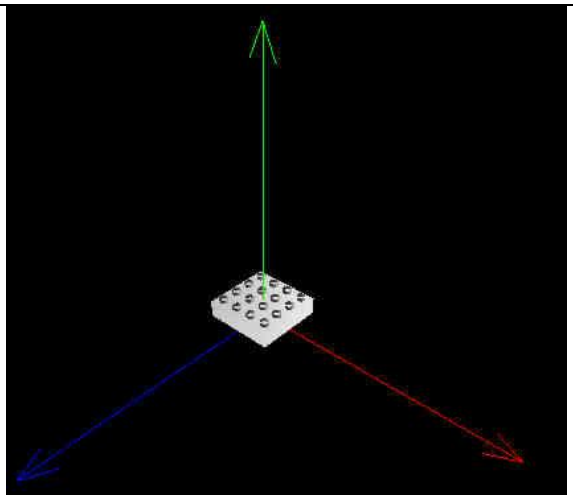
Now that we have a Lego piece with dimension (width and length) 2x2, we can make use of it to build pieces of other larger dimensions. Below is an example, with the output as shown if used.

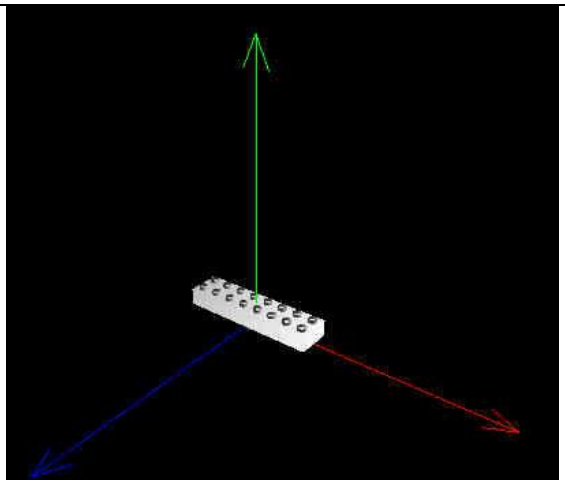
```
void MyLego::draw2x4Lego()
{
 glPushMatrix();
   glTranslatef(-1.0f, 0.0f, 0.0f);
   draw2x2Lego();
   glTranslatef(2.0f, 0.0f, 0.0f);
   draw2x2Lego();
 glPopMatrix();
}

//comment the unused lines to show only
//the piece you are constructing
void MyLego::draw()
{
 //draw1();
 //draw2();
 ....
 glColor3f(1.0f, 1.0f, 1.0f);
 //draw2x2Lego();
 draw2x4Lego();
 ....
}
```

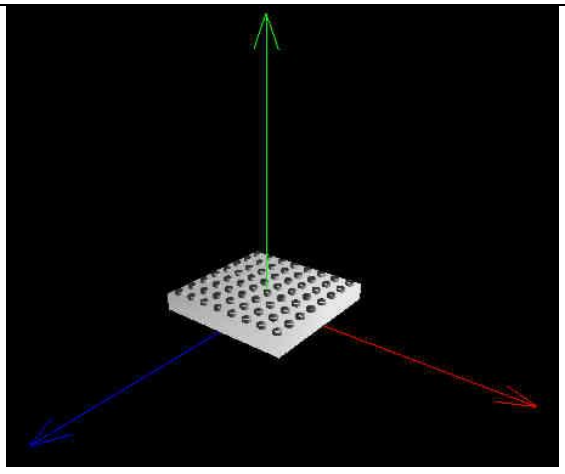


Take note again that `glPushMatrix()` and `glPopMatrix()` are being used. Take note also that the function does not set the color of the piece, this would need to be set before calling the function.

Your task now is to complete the functions below:

`void MyLego::draw4x4Lego()`
`{`
`  //fill this...`
`}`
	

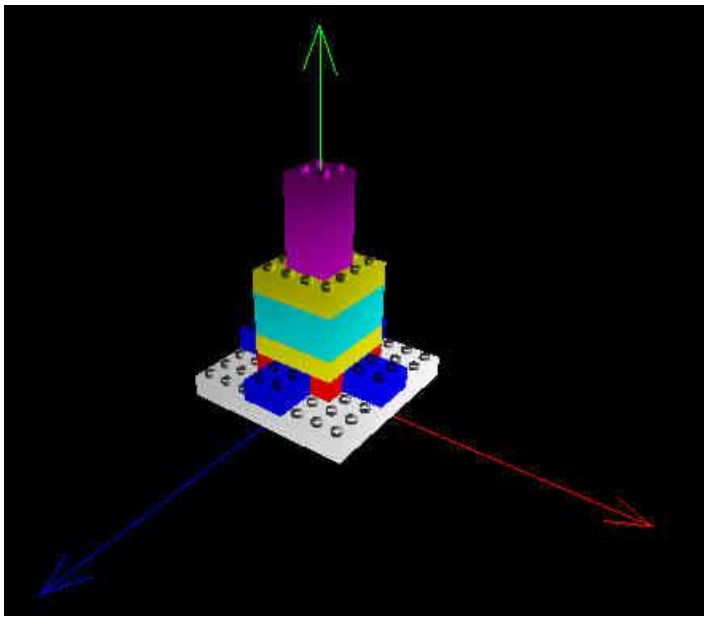
`void MyLego::draw2x8Lego()`
`{`
`  //fill this...`
`}`
	

`void MyLego::draw8x8Lego()`
`{`
`  //fill this...`
`}`
	

By now, you should know how to write higher-level functions (to create more complex objects) by making use of the lower-level functions that you have created earlier for smaller objects. This is the basic technique of modelling objects **HIERARCHICALLY**. This technique will later (in future labs) be useful to create animations on specific parts of an object.

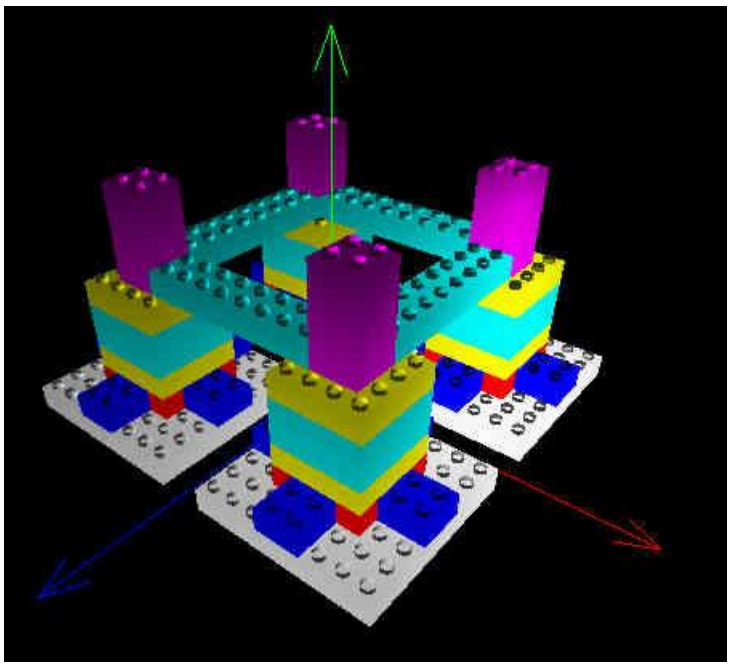
Task 2B: Modelling more complex objects

Based on all that you have learned so far, you are to complete the `MyLego::drawOneTower()` function to build a single Lego tower as shown below:



Task 2C: Modelling even more complex and larger objects

After you have done the above, your task now is to complete the `MyLego::drawTowers()` function to draw the following output:



It may look complicated, but actually it is very simple IF (ONLY IF) you have completed all the tasks before this.

HAVE FUN PLAYING WITH LEGO!!

Extra Exercises

VERY IMPORTANT

BEFORE coming to the next lab, you MUST:

1. COMPLETE all the exercises in this lab!

2. DO the extra exercises below (if there are any)!

3. READ the lab instructions!

4. PREPARE files for next lab!

1. Complete the exercises as your homework if you have not completed them.

2. Another transformation which we have not covered here is `glScalef(...)`, modify your program in Task 2 so that you can have towers of different sizes, this is achieved by using the `glScalef(...)` function. Make sure you utilize the `glPushMatrix()` and `glPopMatrix()` functions, or else things can get really weird!

3. A best way to improve your skill in what you have learned is to create more examples of your own. How about a city of towers? !!

* END OF LAB *